

Secure Code Assessment Report

CodSoft Cyber Security Internship – Task 3

Prepared by: Dhruvi Pramod Jondhalekar

1. Overview

This report documents a manual security review of a sample Python-based user login application (vulnerable_login.py). The purpose of this assessment is to identify common security weaknesses in the source code, explain the risks each one poses, and recommend secure coding practices to fix them.

Application Reviewed:

Name: Simple User Login System

Language: Python 3

Functionality: User registration and login using a local SQLite database

2. Methodology

The source code was reviewed manually, line by line, to identify security anti-patterns commonly seen in real-world applications. Each flaw found is documented below with the vulnerable code, an explanation of the issue, the associated risk, and a recommended fix.

3. Findings Summary

#	Vulnerability	Severity	Category
1	Hardcoded credentials / secret keys	High	Sensitive Data Exposure
2	Plain text password storage	Critical	Sensitive Data Exposure
3	SQL Injection	Critical	Injection
4	Insecure password comparison	Medium	Cryptographic Weakness
5	Missing input validation	High	Improper Input Handling

4. Detailed Findings

Flaw 1: Hardcoded Credentials and Secret Keys

Vulnerable Code:

```
ADMIN_PASSWORD = "admin123"  
SECRET_API_KEY = "sk_live_51Hy8h2KZ9d8fA0bC1x2Y3z4"
```

Explanation:

Sensitive values such as passwords and API keys are written directly into the source code as plain strings.

Risk:

If this code is ever shared, committed to a public repository (e.g. GitHub), or viewed by an unauthorized person, these secrets are immediately exposed. Attackers actively scan public repositories for exactly this kind of exposed credential.

Recommended Fix:

Store secrets in environment variables or a secrets manager, never in source code. Example using environment variables:

```
import os  
ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD")  
SECRET_API_KEY = os.environ.get("SECRET_API_KEY")
```

Flaw 2: Plain Text Password Storage

Vulnerable Code:

```
cursor.execute(  
    f"INSERT INTO users (username, password) VALUES ('{username}', '{password}')"  
)
```

Explanation:

User passwords are stored in the database exactly as entered, with no encryption or hashing applied.

Risk:

If the database is ever stolen or leaked, every user's real password is immediately readable. Since many people reuse passwords across sites, this can lead to account takeovers on other platforms too.

Recommended Fix:

Hash passwords using a strong algorithm (e.g. bcrypt) before storing them, and

compare hashes on login rather than raw passwords:

```
import bcrypt
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
# store 'hashed' in the database instead of the raw password
```

Flaw 3: SQL Injection

Vulnerable Code:

```
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
cursor.execute(query)
```

Explanation:

User-supplied input (username and password) is inserted directly into a SQL query string using string formatting.

Risk:

An attacker can manipulate the query by entering specially crafted input, such as ' OR '1'='1 as the username. This can bypass authentication entirely, allowing login without valid credentials, or let an attacker read/modify/delete data in the database.

Recommended Fix:

Use parameterized queries, which treat user input strictly as data, never as part of the SQL command itself:

```
cursor.execute(
    "SELECT * FROM users WHERE username = ? AND password = ?",
    (username, password)
)
```

Flaw 4: Insecure Password Comparison

Vulnerable Code:

```
if password == ADMIN_PASSWORD:
```

Explanation:

The password is compared using Python's standard equality operator (==), which stops comparing as soon as it finds a mismatched character.

Risk:

This creates a timing side-channel: an attacker measuring how long each login attempt takes can, in theory, guess the password one character at a time. This is known as a timing attack.

Recommended Fix:

Use a constant-time comparison function so the comparison always takes the same amount of time regardless of where the mismatch occurs:

```
import hmac
if hmac.compare_digest(password, ADMIN_PASSWORD):
```

Flaw 5: Missing Input Validation

Vulnerable Code:

```
username = input("Enter username: ")
password = input("Enter password: ")
```

Explanation:

User input is accepted with no checks on length, format, or allowed characters before being used elsewhere in the program.

Risk:

Unvalidated input can lead to multiple downstream issues, including enabling the SQL Injection described above, unexpected crashes from malformed input, or resource exhaustion from excessively long input.

Recommended Fix:

Validate all user input before use: reject empty values, enforce reasonable length limits, and restrict to expected characters:

```
if not username or len(username) > 50 or not username.isalnum():
    raise ValueError("Invalid username")
```

5. Conclusion

This review identified five security weaknesses in the sample login application, ranging from Medium to Critical severity. The most serious issues — SQL Injection and plain text password storage — could allow an attacker to fully bypass authentication or expose every user's password if the database were compromised. Applying the recommended fixes (parameterized queries, password hashing, secret management via environment variables, constant-time comparison, and input validation) would bring this application in line with standard secure coding practices.